GUÍA DE ESTUDIO

Data Warehouse y Analítica de Datos con SQL

Apuntes en español que recorren, tema por tema, un proyecto completo: construir un data warehouse con arquitectura Medallion y luego explotarlo con consultas analíticas. Explicaciones, diagramas y ejemplos SQL redactados de forma independiente.

Fuente de los temas: curso *SQL Data Warehouse Project* y *SQL Data Analytics Project* de Baraa Khatib Salkini (Data with Baraa). Esta guía no reproduce su material; para las láminas y videos originales, consulta el curso.

Contenido

Parte 1 · Data Warehouse	Parte 2 · Analítica
1. Tres tipos de proyecto SQL	12. Dimensiones vs. medidas
2. Qué es un DWH y por qué existe	13. Exploración de dimensiones
3. Arquitecturas de datos	14. Exploración de fechas
4. Capas Bronze, Silver y Gold	15. Exploración de medidas
5. Separación de responsabilidades	16. Magnitud
6. ETL y ELT	17. Cambio en el tiempo
7. Modelado: estrella y copo de nieve	18. Análisis acumulado
8. Integración de datos	19. Análisis de desempeño
9. Catálogo de datos	20. Parte del todo
10. Git y control de versiones	21. Segmentación
11. BULK INSERT vs. INSERT	Anexo: tablas de ejemplo

Convención de los ejemplos. Las consultas usan SQL Server (T-SQL) y un modelo de ejemplo propio: `gold.fact_ventas`, `gold.dim_clientes` y `gold.dim_productos`. Su estructura está en el anexo.

Parte 1 · Construir el Data Warehouse

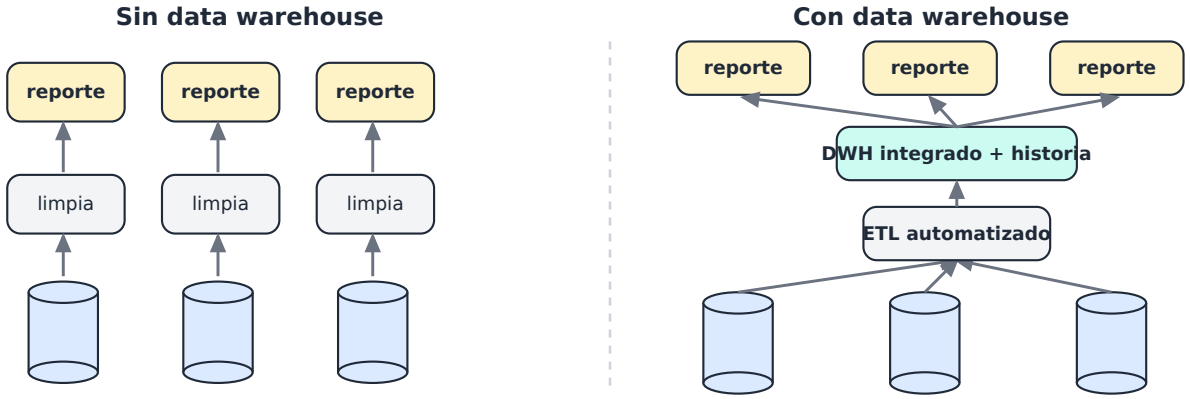
1. Tres tipos de proyecto SQL

Un mismo dominio de datos suele pasar por tres etapas, cada una con un propósito y un nivel de dificultad distinto:

Tipo	Propósito	Técnicas típicas
Data warehousing	Organizar, estructurar y preparar los datos para que otros los usen.	ETL/ELT, arquitectura, integración, limpieza, carga y modelado
Análisis exploratorio (EDA)	Entender qué hay en los datos: rangos, valores, calidad.	Consultas básicas, perfilado, agregaciones simples, subconsultas
Analítica avanzada	Responder preguntas concretas del negocio.	Consultas complejas, funciones de ventana, CTE, subconsultas, reportes

2. Qué es un data warehouse y por qué existe

Un **data warehouse (DWH)** es una base de datos diseñada para análisis: reúne información de varios sistemas, la deja limpia y consistente, y conserva su historia. No reemplaza a los sistemas operativos (ERP, CRM, e-commerce); los alimenta hacia un lugar común.



Sin DWH, cada analista extrae y limpia los datos por su cuenta para cada reporte. El trabajo se repite, las cifras no cuadran entre áreas y el proceso es lento. Con un DWH, la limpieza se hace una sola vez en un proceso automatizado; los analistas consumen una fuente confiable y los reportes salen en horas, no días. Además, el DWH escala a volúmenes grandes y guarda el historial que los sistemas fuente suelen sobrescribir.

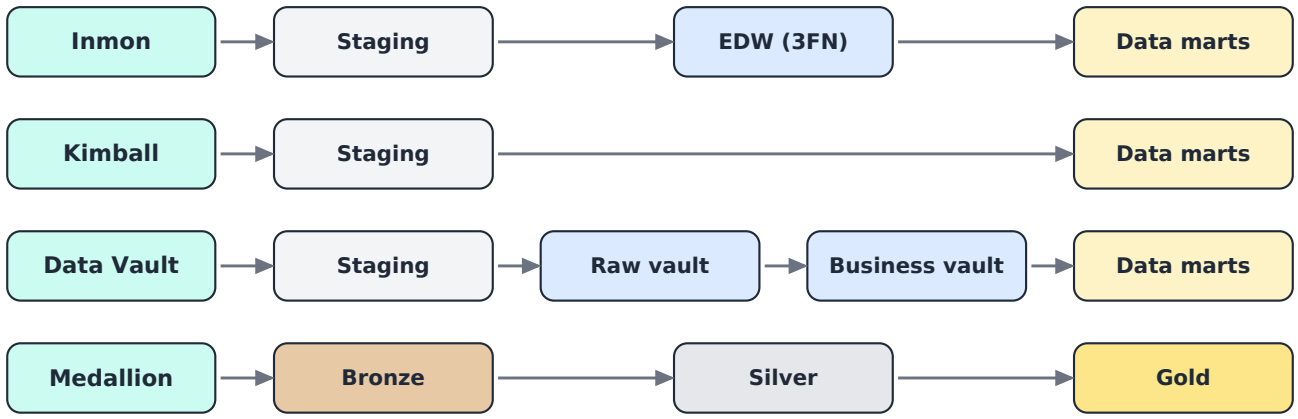
Una forma de pensarlo: los sistemas fuente son las facturas y recibos sueltos de una empresa; el DWH es la contabilidad ya registrada en cuentas ordenadas; los reportes son los estados financieros. Nadie arma un balance directo desde la caja de recibos.

3. Arquitecturas de datos

Hay cuatro grandes familias de arquitectura:

Arquitectura	Idea central
Data warehouse	Datos estructurados, limpios y modelados para reportes.
Data lake	Almacena cualquier dato (estructurado o no) en bruto y barato; se modela al leer.
Data lakehouse	Combina el almacenamiento flexible del lake con la gestión y rendimiento del DWH.
Data mesh	Descentraliza: cada dominio del negocio es dueño de sus propios productos de datos.

Dentro del data warehouse existen cuatro enfoques clásicos para organizar las capas:



Enfoque	Cómo funciona	Cuándo encaja
Inmon	Primero un DWH empresarial normalizado (3FN); de ahí salen data marts por área.	Organizaciones grandes que priorizan una sola versión de la verdad.
Kimball	Va directo de staging a data marts dimensionales (esquema estrella).	Equipos que necesitan resultados rápidos para BI.
Data Vault	Separa una capa cruda (hubs, links, satellites) de otra con reglas de negocio.	Muchas fuentes cambiantes y fuerte necesidad de auditoría.
Medallion	Tres capas progresivas: Bronze, Silver y Gold.	Proyectos modernos y fáciles de explicar; es el que usa este proyecto.

4. Capas Bronze, Silver y Gold



	Bronze	Silver	Gold
Qué contiene	Datos tal como llegan de la fuente	Datos limpios y estandarizados	Datos listos para el negocio
Para qué sirve	Trazabilidad y depuración: siempre se puede volver al origen	Capa intermedia que prepara el análisis	Consumo directo en reportes y analítica
Objeto	Tablas	Tablas	Vistas
Carga	Completa: truncar e insertar	Completa: truncar e insertar	No se carga; las vistas calculan al consultar
Transformación	Ninguna	Limpieza, estandarización, normalización, columnas derivadas, enriquecimiento	Integración, agregación y reglas de negocio
Modelado	Ninguno	Ninguno	Esquema estrella, tablas agregadas o planas
Quién la usa	Ingenieros de datos	Ingenieros y analistas	Analistas y usuarios de negocio

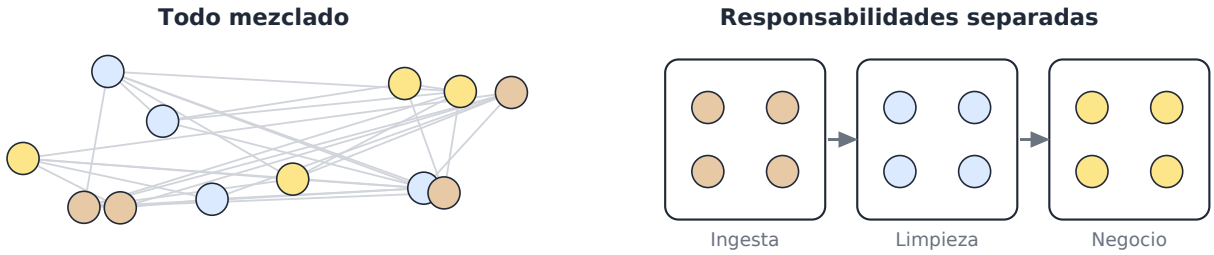
Regla práctica: Bronze nunca se corrige (si se borra, se pierde la evidencia); Silver corrige pero no inventa reglas de negocio; Gold aplica las reglas y le habla al usuario final.

Ejemplo de transformación en Silver: estandarizar un código y derivar una columna.

```
INSERT INTO silver.crm_clientes (cliente_id, nombre, genero, fecha_carga)
SELECT
    cliente_id,
    TRIM(nombre)                                    AS nombre,
    CASE UPPER(TRIM(genero))
         WHEN 'F' THEN 'Femenino'
         WHEN 'M' THEN 'Masculino'
         ELSE 'n/d' END                             AS genero,
    GETDATE()                                       AS fecha_carga
FROM bronze.crm_clientes
WHERE cliente_id IS NOT NULL;
```

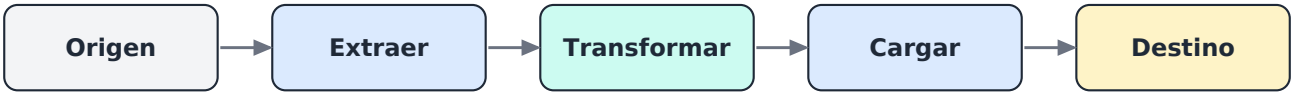
5. Separación de responsabilidades

El principio de **separación de responsabilidades** (*separation of concerns*) dice que cada componente debe hacer una sola cosa y no meterse en la tarea de otro. En un DWH, cada capa tiene un trabajo único: si la limpieza ocurre en Silver, Gold no vuelve a limpiar. Así los errores se localizan rápido y un cambio en una capa no rompe las demás.



6. ETL y ELT

ETL significa extraer (leer de la fuente), transformar (limpiar, unir, calcular) y cargar (escribir en el destino). La extracción puede ser completa o incremental; la carga puede reemplazar todo o agregar lo nuevo.

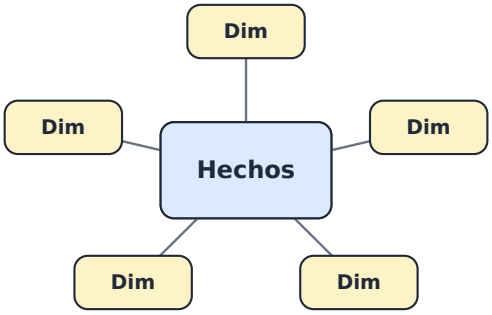


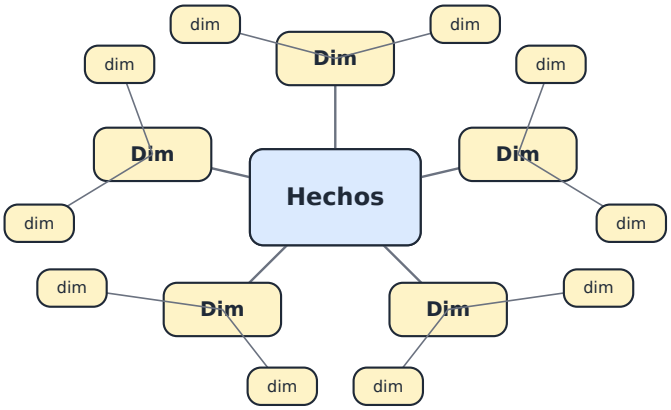
En ELT se carga primero y se transforma dentro del destino.

En una arquitectura por capas no siempre ocurren los tres pasos en cada salto. En Medallion, por ejemplo: de la fuente a Bronze solo se extrae y carga (EL); de Bronze a Silver hay extracción, transformación y carga (ETL); y hacia Gold a menudo solo se transforma, porque las vistas leen de Silver sin copiar datos (T).

7. Modelado de datos: estrella y copo de nieve

El modelado dimensional separa los datos en dos tipos de tablas. La tabla de **hechos** registra eventos medibles (cada venta, cada pago) con sus métricas y llaves. Las tablas de **dimensiones** describen el contexto: quién, qué, dónde, cuándo.



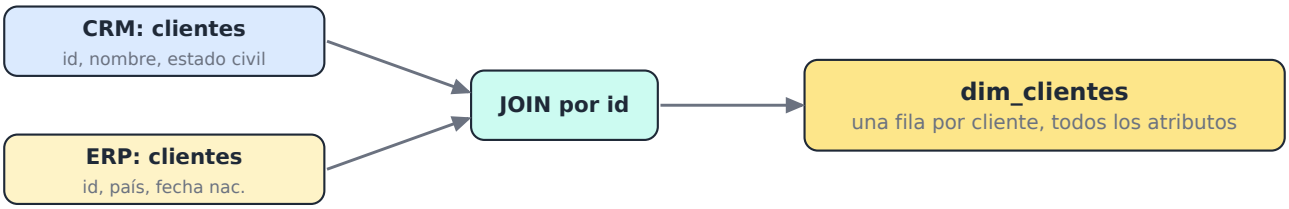


Estrella **Copo de nieve**

	Estrella	Copo de nieve
Estructura	Hechos al centro y dimensiones planas alrededor	Las dimensiones se dividen en sub-dimensiones normalizadas
Consultas	Pocos JOIN, simples y rápidas	Más JOIN, más complejas
Almacenamiento	Algo de redundancia en dimensiones	Menos redundancia
Uso habitual	BI y reportes (Power BI la prefiere)	Dimensiones muy grandes o jerárquicas

8. Integración de datos

Es común que la misma entidad viva en varios sistemas, cada uno con atributos distintos. Integrar es unirlos en una sola tabla coherente, decidiendo qué fuente manda cuando dos dicen cosas distintas.




```
CREATE VIEW gold.dim_clientes AS
SELECT
    ROW_NUMBER() OVER (ORDER BY c.cliente_id)    AS cliente_key,  -- llave sustituta
    c.cliente_id,
    c.nombre,
    COALESCE(NULLIF(c.genero, 'n/d'), e.genero, 'n/d') AS genero, -- el CRM manda
    e.pais,
    e.fecha_nacimiento
FROM silver.crm_clientes c
LEFT JOIN silver.erp_clientes e ON e.cliente_id = c.cliente_id;
```

9. Catálogo de datos

Un DWH bien construido no sirve si nadie sabe qué significa cada columna. El **catálogo de datos** documenta cada tabla y campo: descripción, tipo, origen y reglas. Con él, quien escribe consultas, quien arma un tablero en Power BI y quien entrena un modelo entienden lo mismo sin preguntarle al ingeniero.

Columna	Tipo	Descripción
cliente_key	INT	Llave sustituta generada en el DWH; se usa para unir con los hechos.
cliente_id	INT	Identificador original del cliente en el CRM.
pais	NVARCHAR(50)	País de residencia según el ERP; 'n/d' si no se conoce.

10. Git y control de versiones

El código del DWH (scripts DDL, procedimientos de carga, pruebas, documentación) se guarda en un repositorio Git, por ejemplo en GitHub. Cada cambio queda como una versión; cada persona sube sus cambios (*push*) y descarga los de los demás (*pull*). Una estructura típica del repositorio:

```
datasets/        archivos CSV de las fuentes
docs/            diagramas, catálogo de datos, convenciones de nombres
scripts/
  bronze/        DDL y procedimiento de carga de Bronze
  silver/        DDL y procedimiento de carga de Silver
  gold/          vistas del modelo estrella
tests/           consultas de control de calidad
README.md
```

11. BULK INSERT vs. INSERT

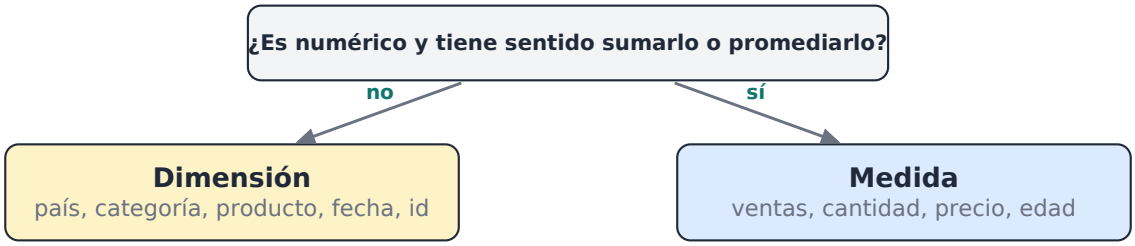
Para cargar archivos CSV o TXT a una tabla, **BULK INSERT** lee el archivo completo en una sola operación, lo que es mucho más rápido que un **INSERT** por fila. Por eso es el método natural para llenar Bronze con carga completa (truncar y volver a insertar).

```
TRUNCATE TABLE bronze.crm_clientes;

BULK INSERT bronze.crm_clientes
FROM 'C:\datos\crm\clientes.csv'
WITH (FIRSTROW = 2, FIELDTERMINATOR = ',', TABLOCK);
```

12. Dimensiones vs. medidas

Antes de analizar, clasifica cada columna. La pregunta clave tiene dos partes: ¿es un número? y ¿tiene sentido agregarlo? Un id es numérico, pero sumar ids no significa nada: es una dimensión.



Casi todo análisis se reduce a una medida agregada, cortada por una o varias dimensiones. Los patrones que siguen son variaciones de esa idea.

#	Patrón	Fórmula general	Herramienta SQL
13	Exploración de dimensiones	Valores únicos de una dimensión	DISTINCT
14	Exploración de fechas	Rango de una fecha	MIN, MAX, DATEDIFF
15	Exploración de medidas	Una métrica total	SUM, AVG, COUNT
16	Magnitud	Medida por dimensión	GROUP BY
17	Cambio en el tiempo	Medida por fecha	GROUP BY + funciones de fecha
18	Acumulado	Medida acumulada por fecha	Funciones de ventana
19	Desempeño	Medida actual vs. objetivo	Funciones de ventana, LAG
20	Parte del todo	Medida / total × 100	Ventana sobre el total
21	Segmentación	Medida por rangos de otra medida	CASE WHEN

13. Exploración de dimensiones

Lista los valores posibles de una dimensión para conocer su granularidad y detectar errores (mayúsculas mezcladas, valores vacíos, duplicados con espacios).

```
SELECT DISTINCT pais FROM gold.dim_clientes ORDER BY pais;

SELECT DISTINCT categoria, subcategoria, nombre_producto
FROM gold.dim_productos
ORDER BY categoria, subcategoria;
```

14. Exploración de fechas

Identifica los límites temporales de los datos: desde cuándo y hasta cuándo hay registros, y cuánto tiempo cubren. También sirve para conocer rangos de edad de clientes.

```
SELECT
    MIN(fecha_pedido)                                    AS primer_pedido,
    MAX(fecha_pedido)                                    AS ultimo_pedido,
    DATEDIFF(MONTH, MIN(fecha_pedido), MAX(fecha_pedido)) AS meses_de_historia
FROM gold.fact_ventas;

SELECT
    DATEDIFF(YEAR, MAX(fecha_nacimiento), GETDATE()) AS cliente_mas_joven,
    DATEDIFF(YEAR, MIN(fecha_nacimiento), GETDATE()) AS cliente_mayor
FROM gold.dim_clientes;
```

15. Exploración de medidas

Calcula las cifras principales del negocio en un solo número cada una. Juntas forman un reporte de indicadores clave que sirve de referencia para el resto del análisis.

```
SELECT 'Ventas totales'    AS indicador, SUM(monto)              AS valor FROM gold.fact_ventas
UNION ALL
SELECT 'Unidades vendidas',              SUM(cantidad)                   FROM gold.fact_ventas
UNION ALL
SELECT 'Precio promedio',                AVG(precio)                     FROM gold.fact_ventas
UNION ALL
SELECT 'Pedidos',                        COUNT(DISTINCT numero_pedido)   FROM gold.fact_ventas
UNION ALL
SELECT 'Clientes con compra',            COUNT(DISTINCT cliente_key)     FROM gold.fact_ventas;
```

16. Magnitud

Compara el tamaño de una medida entre los valores de una dimensión: ventas por país, unidades por categoría, precio promedio por producto, pedidos por cliente. Se visualiza bien con barras ordenadas.

```
SELECT p.categoria, SUM(f.monto) AS ventas
FROM gold.fact_ventas f
JOIN gold.dim_productos p ON p.producto_key = f.producto_key
GROUP BY p.categoria
ORDER BY ventas DESC;
```

17. Cambio en el tiempo

Agrega una medida por periodo (año, mes) para ver tendencias y estacionalidad. Se visualiza con líneas.

```
SELECT
    DATETRUNC(MONTH, fecha_pedido) AS mes,
    SUM(monto)                     AS ventas,
    COUNT(DISTINCT cliente_key)    AS clientes
FROM gold.fact_ventas
WHERE fecha_pedido IS NOT NULL
GROUP BY DATETRUNC(MONTH, fecha_pedido)
ORDER BY mes;
```

18. Análisis acumulado

Suma o promedia progresivamente a lo largo del tiempo: el total acumulado muestra cómo crece el negocio; el promedio móvil suaviza el ruido mensual. Se resuelve con funciones de ventana sobre una agregación previa.

```
WITH mensual AS (
    SELECT DATETRUNC(MONTH, fecha_pedido) AS mes, SUM(monto) AS ventas
    FROM gold.fact_ventas
    WHERE fecha_pedido IS NOT NULL
    GROUP BY DATETRUNC(MONTH, fecha_pedido)
)
SELECT
    mes,
    ventas,
    SUM(ventas) OVER (PARTITION BY YEAR(mes) ORDER BY mes) AS acumulado_anual,
    AVG(ventas) OVER (ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS promedio_movil_3m
FROM mensual;
```

19. Análisis de desempeño

Compara el valor actual con una referencia: el promedio, el periodo anterior o el mínimo. La diferencia dice si un elemento está por encima o por debajo de lo esperado.

```
WITH anual AS (
    SELECT YEAR(f.fecha_pedido) AS anio, p.nombre_producto, SUM(f.monto) AS ventas
    FROM gold.fact_ventas f
    JOIN gold.dim_productos p ON p.producto_key = f.producto_key
    WHERE f.fecha_pedido IS NOT NULL
    GROUP BY YEAR(f.fecha_pedido), p.nombre_producto
)
SELECT
    anio, nombre_producto, ventas,
    ventas - AVG(ventas) OVER (PARTITION BY nombre_producto)                 AS vs_promedio,
    ventas - LAG(ventas) OVER (PARTITION BY nombre_producto ORDER BY anio)   AS vs_anio_anterior,
    CASE WHEN ventas > AVG(ventas) OVER (PARTITION BY nombre_producto)
         THEN 'Sobre el promedio' ELSE 'Bajo el promedio' END                AS estado
FROM anual
ORDER BY nombre_producto, anio;
```

20. Parte del todo

Expresa cada segmento como porcentaje del total para ver qué categorías o países pesan más. Fórmula: (medida del segmento / medida total) × 100.

```
WITH por_categoria AS (
    SELECT p.categoria, SUM(f.monto) AS ventas
    FROM gold.fact_ventas f
    JOIN gold.dim_productos p ON p.producto_key = f.producto_key
    GROUP BY p.categoria
)
SELECT
    categoria,
    ventas,
    CAST(100.0 * ventas / SUM(ventas) OVER () AS DECIMAL(5,2)) AS pct_del_total
FROM por_categoria
ORDER BY ventas DESC;
```

21. Segmentación

Convierte una medida en categorías (bajo, medio, alto) y cuenta cuántos elementos caen en cada una: productos por rango de costo, clientes por edad o por gasto. Es medida por medida, y se resuelve con CASE WHEN.

```
WITH gasto AS (
    SELECT cliente_key,
           SUM(monto)                                          AS total_gastado,
           DATEDIFF(MONTH, MIN(fecha_pedido), MAX(fecha_pedido)) AS meses_activo
    FROM gold.fact_ventas
    GROUP BY cliente_key
)
SELECT segmento, COUNT(*) AS clientes
FROM (
    SELECT CASE
             WHEN meses_activo >= 12 AND total_gastado >  5000 THEN 'VIP'
             WHEN meses_activo >= 12                           THEN 'Regular'
             ELSE 'Nuevo' END AS segmento
    FROM gasto
) s
GROUP BY segmento
ORDER BY clientes DESC;
```

Anexo · Tablas de ejemplo usadas en la guía

Tabla	Columnas	Granularidad
gold.fact_ventas	numero_pedido, producto_key, cliente_key, fecha_pedido, fecha_envio, monto, cantidad, precio	Una fila por línea de pedido
gold.dim_clientes	cliente_key, cliente_id, nombre, genero, pais, fecha_nacimiento	Una fila por cliente
gold.dim_productos	producto_key, producto_id, nombre_producto, categoria, subcategoria, costo	Una fila por producto

Checklist del proyecto

Etapa	Entregable
Diseño	Diagrama de arquitectura, convención de nombres, repositorio Git
Bronze	Tablas crudas + procedimiento de carga con BULK INSERT
Silver	Tablas limpias + procedimiento de carga + pruebas de calidad
Gold	Vistas del esquema estrella + catálogo de datos
Analítica	Consultas EDA y avanzadas, reporte de clientes y productos

Guía de estudio elaborada en español con explicaciones, diagramas y ejemplos propios. Temario basado en los cursos de Baraa Khatib Salkini (Data with Baraa).